Arquitetura Inicial

Projeto	Formandos — gestão administrativa de formaturas e eventos
Marco do cronograma	22/09/2026 (Arquitetura inicial + organização do repositório Git)
Versão	1.0 — elaborada em 26/09/2026
Equipe	Luanderson Arlindo, Luiz Orlando, José Renato e Vinícius
Repositório	https://github.com/luandersonarlindo/Formandos

Este documento registra a arquitetura do Formandos em 26/09/2026. Três decisões marcadas com † delimitam o desenho: a autenticação usa Better Auth com **Google e email e senha** (**D3**), a tabela `membros` permite que um administrador participe de **várias turmas** (**D4**) e a hospedagem é **local**, com o código no GitHub (**D9**). O histórico de como cada uma chegou até aqui está em **Kick-off**.

0. Visão geral

O Formandos é um **monólito web** em **Next.js 16 (App Router)** com **React 19** e **PostgreSQL**. Um único projeto contém as telas, a lógica do servidor e o acesso ao banco. A escolha privilegia **simplicidade**: uma equipe pequena, um só repositório, uma só linguagem (TypeScript) e nenhum serviço extra para manter.

```
Navegador (React 19)
  HTML do servidor + JavaScript (hidratação, Anime.js, Recharts)
       |
       v
Next.js 16 - servidor Node.js
  |- src/proxy.ts ......... sem cookie de sessão? vai para /entrar
  |- Páginas e layouts .... Server Components; checam acesso (dal.ts)
  |- Server Actions ....... src/actions: validam (Zod), checam o
  |                         papel e gravam (SQL)
  '- /api/auth/[...all] ... Better Auth (login, sessão, email, senha)
       |
       v
PostgreSQL (driver pg, SQL parametrizado) - 17 tabelas
  |- Better Auth: usuarios, session, account, verification
  '- Domínio: turmas, membros, catalogos, categorias, enquetes,
     opcoes, votos, duvidas, duvida_upvotes, tarefas, fornecedores,
     programacao, tentativas_convite

Serviços externos: Google OAuth (login) e SMTP/Gmail (emails)
```

1. Decisões de arquitetura

#	Decisão	Alternativas consideradas	Motivo
D1	**Next.js App Router** com Server Components e Server Actions	React puro + API separada; Pages Router	Menos código: a mesma aplicação renderiza, valida e grava, sem uma API REST paralela
D2	**PostgreSQL com SQL puro** (`pg`)	ORM (Prisma, Drizzle)	Aprender SQL de verdade; consultas simples e sob controle; sempre parametrizadas
D3 †	**Better Auth**	Auth.js (NextAuth v5, ainda beta), autenticação própria	Estável, guarda tudo no PostgreSQL, suporta Google e email e senha
D4 †	**Uma tabela `membros`** (usuário, turma, papel)	`turma_id` dentro de `usuarios`	O papel vale **por turma**; permitir várias turmas para administradores foi só remover uma restrição
D5	**URLs sem o id da turma**	`/turmas/[id]/...`	O servidor descobre a turma pelo usuário logado e por um cookie de turma ativa, sempre conferido no banco
D6	**Tailwind CSS 4 + shadcn/ui**	CSS Modules, biblioteca de componentes fechada	Componentes acessíveis e copiados para o projeto; estilo rápido e consistente
D7	**Zod** para validar entradas	Validação manual	Uma fonte da verdade para formato e mensagens de erro
D8	**Recharts** para gráficos	Chart.js, SVG manual	Integra bem com React; gráficos de barras do relatório
D9 †	**Sem hospedagem**: roda localmente; código no GitHub	Vercel ou Render (chegou a ser implementado no Render e depois revertido)	Decisão do grupo; reduz custo e complexidade
D10	**Anime.js** para animações	CSS puro, Framer Motion	API simples; a animação é só um acréscimo, e o conteúdo aparece sem JavaScript

2. Camadas e organização do código

Camada	Pasta	Responsabilidade
Apresentação	`src/app` , `src/components`	Rotas, layouts, telas e componentes; `ui/` (genéricos) e `features/` (de domínio)
Aplicação	`src/actions`	**Server Actions**: as mutações (votar, criar, excluir…). Cada uma valida com Zod, confere o papel e executa o SQL

Camada	Pasta	Responsabilidade
Acesso a dados	`src/lib/*.ts`	Consultas de leitura por área (`votacoes.ts`, `relatorio.ts`, `duvidas.ts`…) e a conexão (`db.ts`)
Segurança	`src/lib/dal.ts`, `src/proxy.ts`	Verificações centralizadas: `exigirSessao`, `exigirMembro`, `exigirAdmin`, `exigirMaster`
Banco	`db/`	`schema.sql`, `apply-schema.mjs` e `seed.mjs` (catálogo padrão lido de `docs/catalogo-enquetes.md`)
Testes	`src/lib/*.test.ts`, `scripts/teste/`	Testes unitários (Vitest) e testes de fluxo no navegador
Documentação	`docs/`, `README.md`	Guia de estudo, catálogo de enquetes e documentos de entrega

Grupos de rotas (pastas entre parênteses, que não aparecem na URL): `(publico)`, `(onboarding)`, `(app)`, `(admin)` e `(master)`. Cada grupo tem seu layout e sua regra de acesso.

3. Modelo de dados

Tabela	Conteúdo	Relações principais
`usuarios`, `session`, `account`, `verification`	Contas, sessões e tokens do Better Auth	`session.userId` e `account.userId` → `usuarios`
`turmas`	Nome, código de convite, data e local do evento	`criado_por` → `usuarios`
`membros`	Vínculo usuário × turma com o **papel** (`admin` ou `participante`)	chave `(turma_id, usuario_id)`
`catalogos`	Conjunto de enquetes; `turma_id` nulo = catálogo padrão global	→ `turmas`
`categorias` → `enquetes` → `opcoes`	Estrutura das votações; `enquetes.tipo` (única ou múltipla) e `opcoes.exclusiva`	encadeadas por chave estrangeira
`votos`	Voto identificado (opção × usuário), guardando a turma	→ `opcoes`, `usuarios`, `turmas`
`duvidas`, `duvida_upvotes`	Perguntas, resposta oficial, status e votos	→ `turmas`, `usuarios`
`tarefas`	Tarefas com responsável, prazo e status	→ `turmas`, `usuarios`
`fornecedores`	Vitrine de terceiros por turma	→ `turmas`
`programacao`	Itens de horário da festa	→ `turmas`

Tabela	Conteúdo	Relações principais
`tentativas_convite`	Códigos errados por usuário, para limitar tentativas	→ `usuarios`

Todas as tabelas do domínio apagam em cascata quando a turma ou o usuário é excluído. O esquema só usa `create table if not exists` e migrações pontuais com `if exists`, para poder ser reaplicado com segurança.

4. Fluxos principais

4.1. Autenticação e acesso

1. A pessoa entra por **Google** ou por **email e senha**; no cadastro por senha, o login só vale depois de **confirmar o email** pelo link.
2. O Better Auth grava a **sessão** no PostgreSQL e devolve um cookie.
3. O `proxy.ts` só confere se **existe** o cookie (checagem rápida). A **validação real** acontece em `lib/dal.ts`, chamada em toda página, layout e Server Action.
4. Sem turma, a pessoa vai a `/convite`; com turma, ao `/dashboard`. A turma em uso vem do cookie `turma_ativa`, **sempre conferido** contra as turmas de que ela é membro.

4.2. Uma ação de escrita (exemplo: votar)

1. O formulário envia os dados para a **Server Action** `votar`.
2. A action chama `exigirMembro()` (sessão e turma) e valida o corpo com **Zod**.
3. Numa **transação**, apaga os votos anteriores do usuário naquela pergunta e grava os novos.
4. A página é revalidada e a tela mostra o voto salvo.

5. Segurança em camadas

Camada	Proteção
Rede	Cabeçalhos HTTP (`X-Content-Type-Options`, `X-Frame-Options: DENY`, `Referrer-Policy`, `Permissions-Policy`) e `X-Powered-By` desligado
Acesso	`proxy.ts` (rápido) + `dal.ts` (definitivo) em páginas e ações; painel master responde 404 para quem não é master
Dados	Toda entrada validada com Zod; SQL sempre parametrizado; cascata nas exclusões
Abuso	Limite de 10 códigos de convite errados a cada 15 minutos por usuário; transação com trava para entrar em turma
Segredos	`.env.local` fora do Git; chaves lidas de variáveis de ambiente; `client_secret_*.json` no `.gitignore`
Privacidade	O relatório mostra só totais; quem votou só aparece para administradores

6. Configuração e ambientes

Variável	Uso
`DATABASE_URL`	Conexão com o PostgreSQL
`BETTER_AUTH_SECRET` e `BETTER_AUTH_URL`	Segredo das sessões e endereço do app
`GOOGLE_CLIENT_ID` e `GOOGLE_CLIENT_SECRET`	Login com Google (opcional: sem elas, o botão some)
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `EMAIL_FROM`	Envio de emails (sem `SMTP_HOST`, o email é escrito no terminal)
`ADMIN_MASTER_EMAILS`	Emails dos administradores master

Ambiente atual: **um só**, o de desenvolvimento local (Node.js e PostgreSQL na máquina). Não há hospedagem.

7. Qualidade

- **Tipos:** TypeScript estrito; `npm run typecheck` gera os tipos das rotas antes de conferir.
- **Testes unitários (Vitest):** regras puras de convite, datas, vínculos e master (20 testes).
- **Testes de fluxo no navegador:** roteiro em `scripts/teste/fluxos.mjs` (59 passos).
- **Auditoria de responsividade:** `scripts/teste/auditoria.mjs`.
- **Pontos ainda ausentes:** ESLint e integração contínua (previstos para a Sprint 2).

8. Riscos e dívidas técnicas

Item	Impacto	Plano
Next.js 16 muda APIs conhecidas (`proxy.ts`, `params` como *Promise*, `error.tsx` com `retry`)	Código de tutoriais antigos quebra	Ler a documentação em `node_modules/next/dist/docs/` antes de codar
SQL escrito à mão em muitos pontos	Repetição e risco de erro de digitação	Testes de fluxo cobrem os caminhos principais; extrair consultas comuns
Sem integração contínua	Erros só aparecem na máquina de quem roda	Adicionar `typecheck` e `npm test` a cada `push` na Sprint 2
Sem hospedagem	Não há teste em ambiente parecido com produção	Documentar o passo a passo local; avaliar hospedagem se o grupo decidir

9. Evolução prevista

Ligar a integração contínua, adicionar ESLint, ampliar os testes de ponta a ponta e, se houver tempo, tema escuro e edição de perguntas (ver *Sprint Backlog*).